Pre-Proposal

CSE333: Introduction to Human-Computer Interaction

Jung WonJun · Know TaeLim · Kang MinJin · Choi YooJin

UNIST

Ulsan, Republic of Korea

Idea 1 · Adaptive Cooking Roadmap

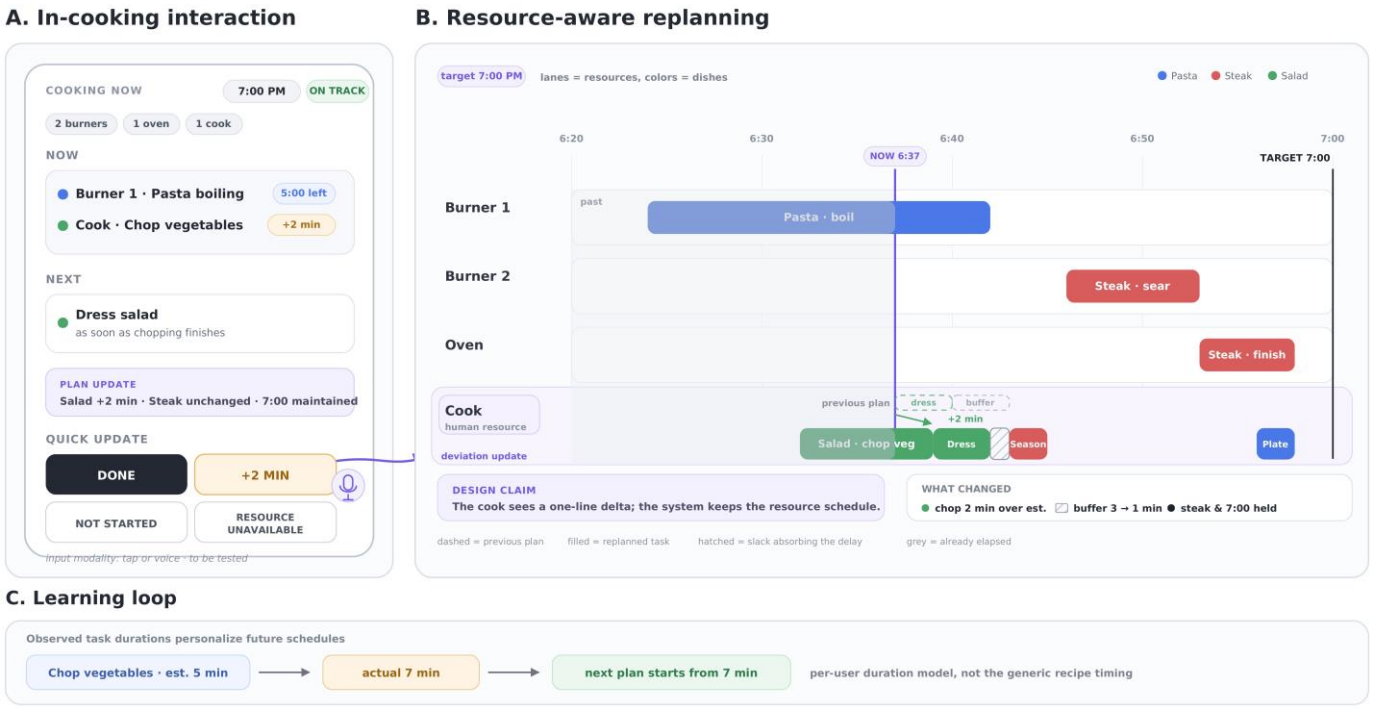
Background & Motivation

- **Why this topic:** preparing a meal usually means several dishes cooked at once — a resource-constrained scheduling problem (shared burners/oven, timing dependencies), not just step-by-step recipe-following
- **Problem:** a static plan goes stale the moment real task durations or resource availability diverge from estimates — but the cook must replan mentally while hands and attention are already occupied
- **Goal:** an interactive cooking assistant that builds a resource-aware, multi-recipe roadmap and lets the cook update it through low-effort input when reality diverges
- Goal and value
  - **Useful** — cuts the mental cost of replanning mid-cook
  - **Interesting** — reframes cooking as ongoing human–system coordination, not one-time optimization
  - **Important** — duration estimates personalize to the individual cook over repeated use

Idea 1 · Adaptive Cooking Roadmap

Conceptual Sketch

The system merges multiple recipes into one resource-aware roadmap, then replans live from low-effort deviation reports (Done / +2 min / Resource unavailable) while learning the cook's real pace over time.



Idea 1 · Adaptive Cooking Roadmap

Implementation & Interaction Scenario

Implementation

- **Hardware:** smartphone/tablet in the kitchen; mic for optional hands-free voice input
- **Software:** React + TypeScript frontend; Python backend for recipe parsing, scheduling, replanning, and logging
- **Algorithm:** tasks modeled with duration, precedence, and resource constraints; on a deviation report the scheduler updates only the remaining tasks, not completed work; observed durations personalize future estimates

Application Scenario

- Jimin cooks pasta, steak, and salad, targeting 7:00 PM with 2 burners
- The system builds one combined roadmap instead of three separate recipes
- Chopping runs long — she taps “+2 min” (or says it); the roadmap replans without moving the 7:00 target
- A burner becomes unavailable — the system replans again around it
- Her actual chop time (7 vs. 5 min estimated) is logged to personalize the next roadmap

Idea 1 · Adaptive Cooking Roadmap

Previous Work

- **Paprika, Umami, Kitchen Stories** — recipe storage, step instructions, timers; Paprika pins/switches active recipes, but still centers on navigating one recipe at a time
- **SideChef** — combines steps from several recipes into one shared timeline
- **Optimal Turkey** — models limited resources (burners/oven/microwave); auto-builds multi-dish schedules and detects resource conflicts
- **Mise & ParallelCook** — generate optimized parallel workflows for preparing multiple dishes
- **SousSmart** — closest prior work: real-time replanning when cooking deviates from the initial timeline

Our contribution

Not just placing recipes on one timeline — we focus on the interaction *around replanning*: how an already-occupied cook reports a deviation with minimal effort, how its consequences are exposed, and how observed performance personalizes future plans.

Idea 2 · Companion Gap

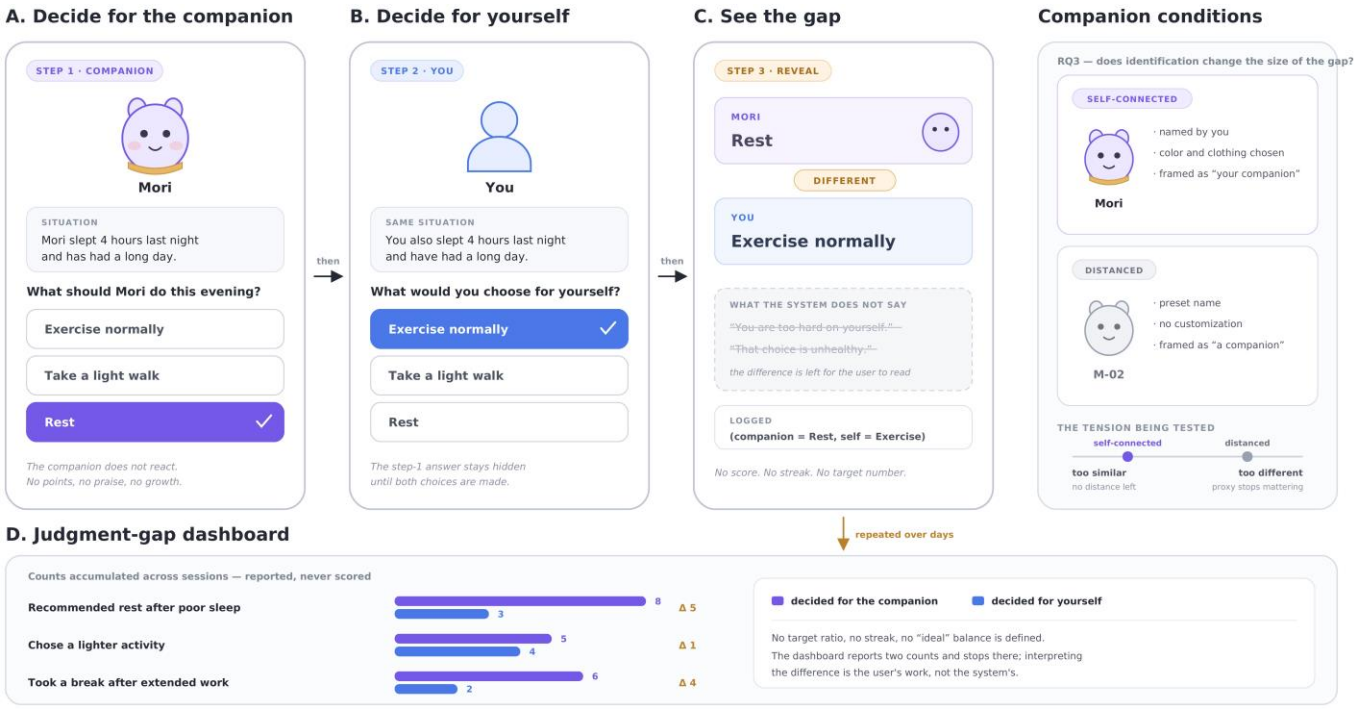
Background & Motivation

- **Why this topic:** most health apps turn behavior into an output — a character the user watches, reflecting what they've already done
- **Problem:** visualizing, rewarding, or gamifying behavior doesn't expose how differently a person judges the same situation for themselves vs. for another entity
- **Goal:** a virtual-companion interface that asks users to decide *for the companion* first, then for themselves — and shows the resulting judgment gap without interpreting it
- Goal and value
  - **Useful** — lightweight self-reflection without the system diagnosing the user
  - **Interesting** — the companion becomes an elicitation interface (a proxy decision target), not a reward/gamification asset
  - **Important** — grounded in psychological-distance / "Solomon's paradox" research; open question of whether a companion creates enough distance to reveal self/other judgment differences

Idea 2 · Companion Gap

Conceptual Sketch

The user first decides for a companion (e.g., Mori) in a health situation, then decides the same situation for themselves; the interface reveals the resulting gap without saying which choice is correct. RQ3 tests self-connected vs. distanced companion conditions.



Idea 2 · Companion Gap

Implementation & Interaction Scenario

Implementation

- **Hardware:** phone/tablet/desktop; later an optional small separate display (e-ink/LCD) for physical separation
- **Software:** React/TypeScript for customization, scenarios, and the dashboard; lightweight backend for condition assignment, decisions, response times, session summaries
- **Algorithm:** logs the ordered pair (companion, self); computes match and disagreement direction; aggregates within-user patterns; scenario order randomized
- **Scope & safety:** no body-weight/shape/disease-based appearance; no photo calorie estimation; companion never deteriorates as punishment; low-risk everyday scenarios only

Application Scenario

- Jimin opens the app and sees Mori, whom she named and customized
- "Mori slept 4 hours, long day — what should Mori do?" — she picks Rest
- Same situation for herself — she picks Exercise normally
- Reveal: "Mori: Rest — You: Exercise normally," with no added judgment
- After several days, the dashboard shows 8 companion-rest vs. 3 self-rest recommendations — she interprets the gap herself

Idea 2 · Companion Gap

Previous Work

- **Samsung Health** — comprehensive tracking across activity/sleep/nutrition/body measurements; role is measurement + interpretation of the user's own data
- **Wrist Doctor 9988** (Seoul City) — health challenges + redeemable points; external-reward model
- **AHealth** — walking/sleep/meals/hydration tracking + "Bio Age" + rewardable points + AI meal-photo feedback
- **Miobi** — closest prior work: meal photos → nutrition insights → a cartoon companion mirroring balance/energy/streaks, but the avatar remains an output the user watches
- **Finch** — a virtual pet grown by completing self-care goals; the user acts to progress the pet (still output-directed)
- **Fish'n'Steps & UbiFit Garden** — earlier HCI systems mapping steps/activity onto a virtual fish / ambient garden (also output-oriented)

Our contribution

The direction of interaction is reversed: the companion's decision is elicited *before* the self decision, making it an input rather than an output. The discrepancy between proxy and self judgments becomes the primary data — not simply a display of behavior.

Idea 3 · The Cost of Overriding

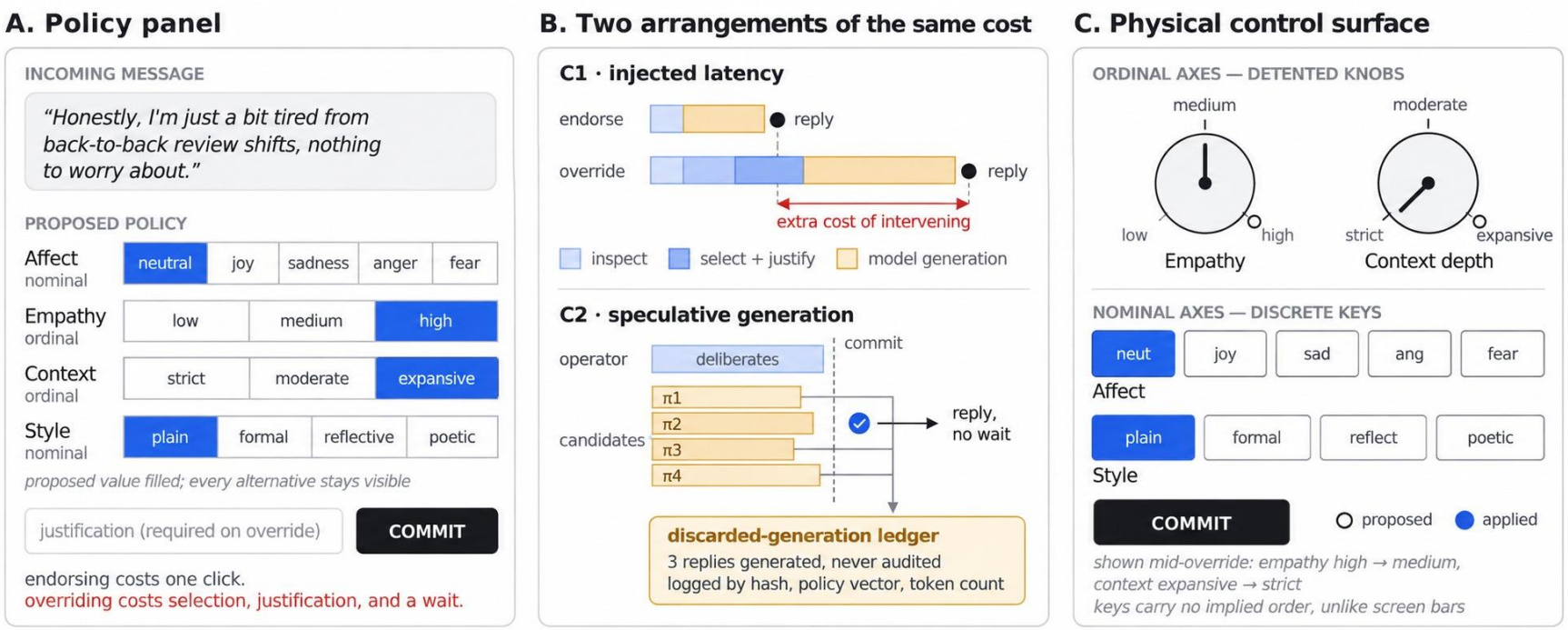
Background & Motivation

- **Why this topic:** giving a supervisor real authority over an LLM's reply means moving intervention before generation — a policy panel the supervisor can accept or override
- **Problem:** accepting is instant; overriding costs reading, selection, justification, and a wait — so a logged "endorsement" can't be told apart from avoidance of that wait, and the chosen policy's length also sets the wait, so supervisors may drift toward whatever is quick to wait for
- **Goal:** treat latency as a manipulable design variable — build a policy interface where intervention cost can be injected, removed (speculative generation), or restructured (a physical control surface), and measure how each changes override behavior, selection time, and the policies chosen
- Goal and value
  - **Useful** — generalizes to any human-in-the-loop gate (agent pause points, review queues, approval workflows) that places a wait behind one branch and not the other
  - **Interesting** — surfaces a structural tension: the fastest way to remove the wait (pre-generating candidates) produces text that was never audited
  - **Important** — the interaction record currently can't distinguish genuine agreement from latency-avoidance, undermining the accountability the interface is meant to provide

Idea 3 · The Cost of Overriding

Conceptual Sketch

A four-axis policy panel lets a supervisor accept or override an LLM's proposed reply behavior before generation. Three conditions rearrange where the cost of overriding falls: injected latency, speculative generation, and a physical control surface.



Idea 3 · The Cost of Overriding

Implementation & Interaction Scenario

Implementation

- **Hardware:** Arduino-class controller with native USB HID (Pro Micro/Teensy) — 2 detented rotary encoders (ordinal axes), 2 button arrays of 5+4 keys (nominal axes), 1 commit key, per-axis LEDs for proposed vs. applied
- **Software:** React/TypeScript operator panel; Python FastAPI backend for policy prediction, generation, and logging; an LLM API; WebSerial/WebHID bridge; SQLite session logs
- **Algorithm:** (1) a latency scheduler with independent per-path delay profiles for endorse vs. override; (2) a speculative routine that ranks and generates the proposal's single-axis neighbors in parallel, logging discarded candidates; (3) timing instrumentation separating supervisor deliberation from model latency

Application Scenario

- Jimin supervises peer-support replies; a message arrives: "Honestly, I'm just a bit tired… nothing to worry about"
- Proposed policy: empathy high, context expansive, affect neutral, style plain
- She judges it as over-reading a mild complaint — turns empathy to medium, context to strict, justifies, commits
- Asymmetric-latency condition: her two corrections cost a noticeable wait; by message 8, the log shows whether she still corrects at the same rate
- Speculative condition: the empathy-medium variant was already generated while she read — the reply appears the instant she commits, and the record notes 3 discarded replies with their hashes

Idea 3 · The Cost of Overriding

Previous Work

- **Miura et al., CHI 2025** — *Understanding and Supporting Formal Email Exchange by Answering AI-Generated Questions*. Their system, ResQ, lets users reply to formal emails by answering short AI-generated questions instead of writing free-form prompts — improving reply efficiency and lowering workload while maintaining email quality.

Our contribution

Like ResQ, we place a lightweight human step between an LLM and its output. But ResQ's questions shape *what gets generated*; our policy panel is a supervisory checkpoint *after* a policy is proposed, and the object of study is the asymmetric latency cost of accepting vs. overriding it — not the intervention step itself.